

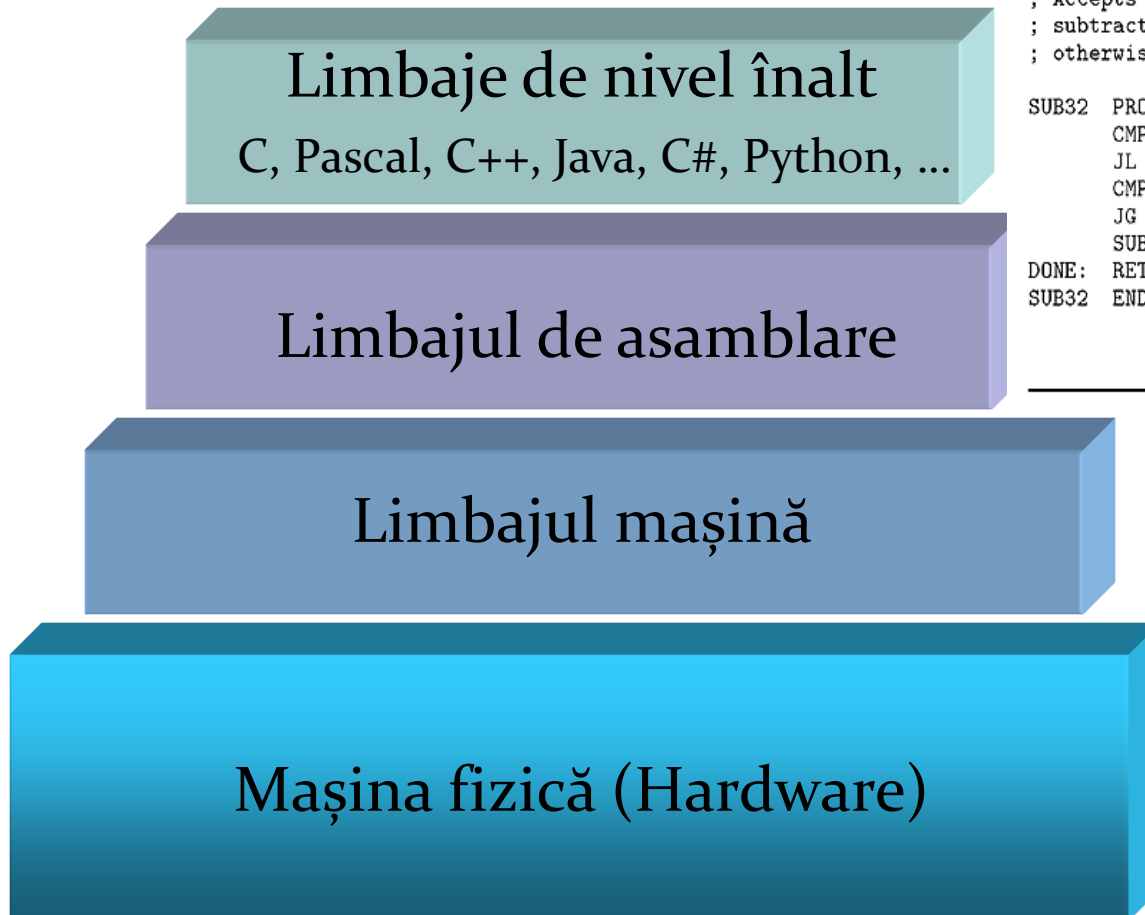
Programare în limbaj de asamblare

Concepte de bază

Liana Boca

Anca Vasilescu

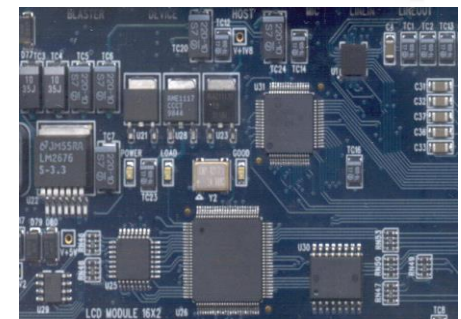
Locul limbajelor în programare



; Example of IBM PC assembly language
; Accepts a number in register AX;
; subtracts 32 if it is in the range 97-122;
; otherwise leaves it unchanged.

```
SUB32 PROC      ; procedure begins here
CMP AX,97      ; compare AX to 97
JL  DONE      ; if less, jump to DONE
CMP AX,122     ; compare AX to 122
JG  DONE      ; if greater, jump to DONE
SUB AX,32      ; subtract 32 from AX
DONE: RET      ; return to main program
SUB32 ENDP     ; procedure ends here
```

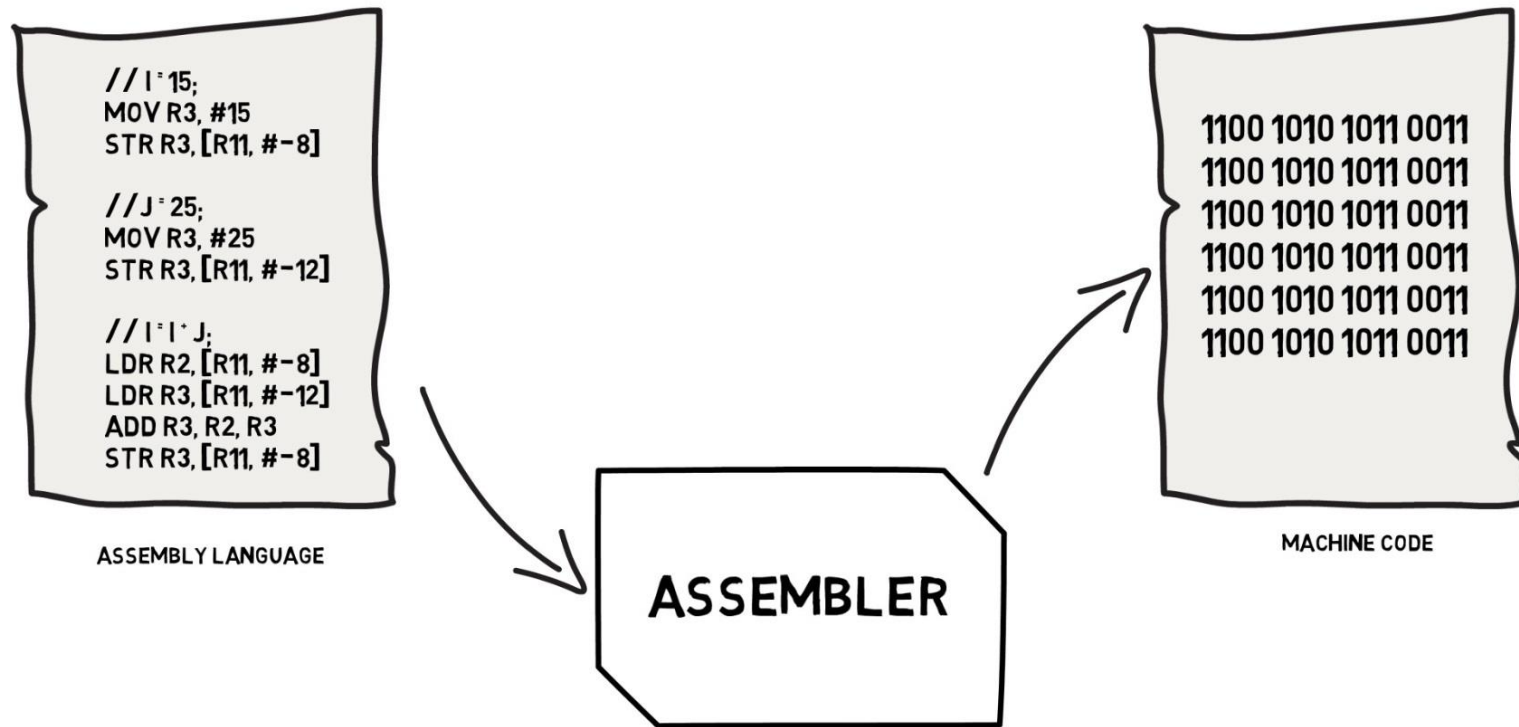
FIGURE 17. Assembly language



Limbajul mașină

- **Singurul** cod pe care procesorul îl poate executa în mod direct este *codul mașină*, scris în *limbajul mașină* al procesorului respectiv
- Acesta este “limbajul natural” al procesorului, **singurul** pe care îl recunoaște și este dependent de arhitectura (alcătuirea hardware) a mașinii (calculatorului) gazdă
- Limbajele mașină sunt mai greu de folosit de către programatori decât limbajele de nivel înalt

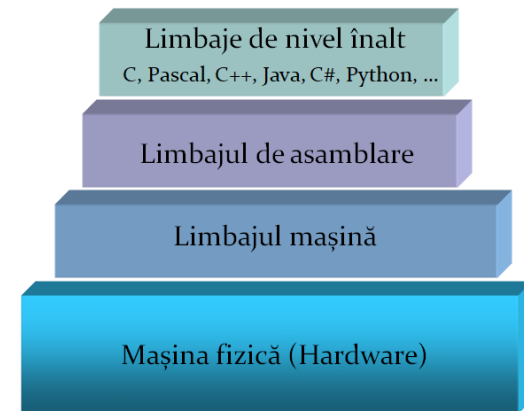
cod în asamblare vs. cod mașină



Ce este limbajul de asamblare?

- Este **forma simbolică** a limbajului recunoscut de mașina fizică (limbajul mașină)
- Folosește simboluri și cuvinte cheie (*încă*) *inteligibile* pentru om, dar **direct translatable** în cod mașină (secvențe de biți de 0 și 1)
- Operează direct cu resursele fizice ale unui calculator: registre, memorie, interfețe de intrare/ieșire, sistem de întreruperi, etc.
- **Important:** indiferent de limbajul folosit pentru scrierea codului, în final, toate *programele executabile* iau forma unui program în limbajul mașină al procesorului pe care se execută

Limbajul de asamblare



- Este următorul nivel de la limbajul mașină
- Oferă programatorului posibilitatea să introducă un cod sursă scris ca text (*încă*) *citit de om*, folosind **nume și simboluri mnemonice**, încă mai accesibil decât secvențele brute (greoaie) de biți ale codului mașină corespunzător
- Este o variantă mult mai puțin obositoare și predispusă la erori decât programarea direct în codul binar al mașinii

Limbajul de asamblare

- Fiecare linie de cod scrisă în limbaj de asamblare reprezintă o instrucțiune pentru procesor.
- **Asamblarea** este etapa de traducere a codului sursă scris în (limbaj de) asamblare în cod mașină.
- Un program de calculator cunoscut sub numele de **asamblator** sau **asamblor** (pe care cineva a trebuit să-l scrie) rezolvă asamblarea, adică traduce instrucțiunile codului scris în limbaj de asamblare (mnemonic) în instrucțiuni corespondente, binare, echivalente.

Computer languages

Machine Language	Assembly Language	High-Level Language
Directly understood by a computer	Assembler → converts to machine language	Compiler (or interpreter) → converts to machine language
	1 assembly language instruction = 1 machine language instruction	1 HLL instruction = many machine language instructions
Not standard (I.e. different machine language for every type of machine)	Not standard (I.e. different assembly language for every type of machine)	Standard (I.e. programs are independent of the machine on which they will be executed)

Generalități

- Codul în asamblare este mai apropiat de codul executabil al programelor, **asamblorul** având **rolul** de a asambla (translata) instrucțiunile din limbaj de asamblare în cod mașină.
- Limbajul de asamblare este din ce în ce mai puțin folosit în scrierea programelor, deoarece compilatoarele de ultima generație au funcțiile cele mai folosite de programatori deja scrise și optimizate în limbaj de asamblare (*ex. memmove, memset / fillchar, etc.*) incluse în unit-uri și biblioteci.

Utilitatea limbajului de asamblare

- **VITEZĂ:** este foarte probabil ca programele scrise în asamblare să fie cele mai rapide
- **SPAȚIU:** programele scrise în asamblare ocupă cel mai puțin spațiu de memorie
- **GRAD de LIBERTATE:** maxim pt. programarea în asamblare
- **CUNOAȘTERE:** contribuie la o mai bună înțelegere a modului de lucru al procesorului, care poate ajuta la scrierea unor programe eficiente în limbajele de nivel înalt
- **asm este un limbaj de programare educațional** [Wikipedia], alături de BASIC, C, C++, PASCAL, LISP, JAVA

Avantajele limbajului de asamblare

- Crește viteza de rulare a programelor.
- Scade dimensiunea codului sursă.
- Ajută programatorul să înțeleagă mai bine cum funcționează microprocesorul și să scrie programe eficiente și în limbaje de programare de nivel înalt.

Dezavantajele limbajului de asamblare

- nu este 100% portabil; un program scris pentru un model de procesor poate să nu fie recunoscut 100% corect de alt procesor;
- un program în asamblare este mai greu de scris și ia mai mult timp decât un program scris într-un HLL;
- un program scris în asamblare este mai greu de depanat;

ASAMBLARE

Elementele limbajului de asamblare

Elementele limbajului de asamblare

- registrele generale și registrul de flag-uri
- tipuri de date

EXAMPLE de programe in limbaj de asamblare

- instrucțiuni
- întreruperi

Hello World

Limbaj de asamblare

HELLO WORLD (1)

TITLE hello

.MODEL small ; Directiva .MODEL este folosită pentru inițializarea
; modelului de memorie, semnificând modul de
; organizare a informației în memoria sistemului

.STACK 100h ; directiva .STACK definește o stivă de 256 octeți,
; suficientă pentru programe mici

.DATA ; Directiva .DATA marchează începutul segmentului
; de date.
; În cazul de față acest segment conține o variabilă
; șir de caractere, Message1, ce reține mesajul care
; va fi afișat în program. Numerele 13, 10 codifică
; „Enter” - (trecerea la rândul următor)

Message1 DB 'Hello! ',13,10, '\$'

HELLO WORLD (2)

```
mov ax,@data ; Primele două instrucțiuni inițializează registrul DS.  
              ; Simbolul @data reprezintă numele segmentului  
              ; creat cu directiva .DATA. Registrul DS va fi  
              ; întotdeauna inițializat pentru fișiere sursă în format  
              ; EXE  
  
mov ds,ax
```

HELLO WORLD (3)

```
mov dx, offset Message1
```

```
mov ah,9h          ; Conținutul variabilei de tip șir de caractere definită  
                   ; în program se afișează pe ecran  
                   ; utilizând funcția DOS cu codul 09h
```

```
int 21h
```

```
mov ah,4ch ; Funcția DOS cu codul 4Ch este utilizată pentru  
           ; terminarea programului și predarea controlului la  
           ; sistem
```

```
int 21h
```

```
END Start
```

Rularea programelor

**Utilizarea macroasamblorului TASM și a
depanatorului TD**

EXEMPLU

Utilizarea macroasamblorului TASM și a depanatorului TD

1. **EDITARE.** Programul în limbaj de asamblare este scris cu un **editor de text** oarecare, devenind **cod (program) sursă** (*source file*).
2. **ASAMBLARE.** După scrierea programului sursă, el este încărcat în **asamblor** (*assembler*), care citește fișierul sursă și produce un **fișier obiect** (*object file*), care reprezintă traducerea programului într-un limbaj mașină. Opțional, asamblorul produce un **fișier listing** (*listing file*).
3. **EDITARE DE LEGĂTURI.** Fișierul obiect este citit de către **linkeditor** (*linker*), care verifică dacă programul conține apeluri de proceduri dintr-o bibliotecă externă (*link library*). Linkeditorul copiază procedurile necesare, le combină cu fișierul obiect, și produce **fișierul executabil** (*executable file*).
4. **EXECUȚIA.** Utilitarul sistemului de operare citește fișierul executabil în memorie și scrie în CPU adresa de pornire a programului, apoi programul începe să se execute.